CSE 222 HW2

Fatih Emre OZAN
2301040040 90

①

a) $2^{2n} + 1 = O(2^n)$

$\lim_{n\to\infty} \left(\frac{2^{2n}}{2^n} + \frac{1}{2^n}\right) = \lim_{n\to\infty} 2^n + \lim_{n\to\infty} 0 = \infty$

so $2^{2n+1} = O(2^n)$ is not true.

b) $2^{2n} + 1 = \Omega(2^n)$ is true condition.

$n^3 - 4n^2 + 7 = \Theta(2^n)$

$\lim_{n\to\infty} \left(\frac{n^3}{2^n} - \frac{4n^2}{2^n} + \frac{7}{2^n}\right)$ ⇒ Exponential functions always grow much faster than polynomial functions ⇒ $\lim_{n\to\infty} (0 - 0 + 0) = 0$

so

$n^3 - 4n^2 + 7 = \Theta(2^n)$ is false.

$n^3 - 4n^2 + 7 = O(2^n)$ is true condition.

c) $n^3(1+\sqrt{n}) = O(n^3 \log n)$

$\lim_{n\to\infty} \left(\frac{n^3(1+\sqrt{n})}{n^3(\log n}\right) = \lim_{n\to\infty} \left(\frac{1+\sqrt{n}}{\log n}\right) = \lim_{n\to\infty} \left(\frac{\sqrt{n}}{\log n}\right) = \infty$

so;

$n^3(1+\sqrt{n}) = O(n^3 \log n)$ is false

$n^3(1+\sqrt{n}) = \Omega(n^3 \log n)$ is true condition.

d) $28n = O(7n^2)$

$\lim_{n\to\infty} \left(\frac{28n}{7n^2}\right) = \lim_{n\to\infty} \left(\frac{4}{n}\right) = 0$

so

$28n = O(7n^2)$ is true condition

e) $n + \log n + 21 = O(7n^2)$

$\lim_{n\to\infty} \left(\frac{n}{7n^2} + \frac{\log n}{7n^2} + \frac{21}{7n^2}\right) = \lim_{n\to\infty} (0 + 0 + 0) = 0$

so $n + \log n + 21 = O(7n^2)$ is true condition.

f) $n^2 + 9n - 13 = \Theta(n^2)$

$\lim_{n\to\infty} \left(\frac{n^2}{n^2} + \frac{9n}{n^2} - \frac{13}{n^2}\right) = \lim_{n\to\infty} (1 + 0 + 0) = 1$

so $n^2 + 9n - 13 = \Theta(n^2)$ is true condition.

Fatih Emre Ozgan
2301040040

② $2n^2, 3n^4, 7n, \log n, 3^n, n!$

For this example we will do a limit comparison.

$$\lim_{n\to\infty}\left(\frac{\log n}{7n}\right)=\lim_{n\to\infty}\left(\frac{\log n}{2n^2}\right)=\lim_{n\to\infty}\left(\frac{\log n}{3n^4}\right)=\lim_{n\to\infty}\left(\frac{\log n}{3^n}\right)=\lim_{n\to\infty}\left(\frac{\log n}{n!}\right)=0$$

Result=0 so $\log n$ grows slower than the other function, meaning its growht rate is less than the other functions.

Other functions $g(n) \Rightarrow \log n = O(g(n))$

$\lim_{n\to\infty}\left(\frac{7n}{2n^2}\right)=\lim_{n\to\infty}\left(\frac{7}{2n}\right)=0 \quad 7n = O(2n^2)$

$\lim_{n\to\infty}\left(\frac{7n}{3n^4}\right)=\lim_{n\to\infty}\left(\frac{7}{3n^3}\right)=0 \quad 7n = O(3n^3)$

$\lim_{n\to\infty}\left(\frac{7n}{3^n}\right)=0$ (exponential growth is faster than linear growth)

$\lim_{n\to\infty}\left(\frac{7n}{n!}\right)=0$ (factorial growth is faster than linear growth)

That is the growth rate of $7n$ is greater than $\log n$ but less than the others.

$\lim_{n\to\infty}\left(\frac{2n^2}{3n^4}\right)=\lim_{n\to\infty}\left(\frac{2}{3n^2}\right)=0$

$\lim_{n\to\infty}\left(\frac{2n^2}{3^n}\right)=0$ (exponential growth is faster than polynomial growth)

$\lim_{n\to\infty}\left(\frac{2n^2}{n!}\right)=0$ (factorial growth is faster than polynomial growth)

in terms of growht rate:

$2n^2 > 7n > \log n$ (from the transactions so far)

$\lim_{n\to\infty}\left(\frac{3n^4}{3^n}\right)=0$ (exponential growth is faster than polynomial growth)

$\lim_{n\to\infty}\left(\frac{3n^4}{n!}\right)=0$ (factorial growth is faster than polynomial growth)

in terms of growht rate:

$3n^4 > 2n^2 > 7n > \log n$ (from the transactions so far)

$\lim_{n\to\infty}\left(\frac{3^n}{n!}\right)=0$ (factorial growth is faster than polynomial growth)

in conclusion

in terms of growth rate; $(2n^2)$

logaritmic growth < linear g. < Quadratic g. < polynomial g < exponential g. < factorial g.

$\log n < 7n < 2n^2 < 3n^4 < 3^n < n!$

Fatih Emre Ogan
230104004090

① In this question, when calculating the worse case complexity, we will find the number of times each step occurs and the worst case scenario result will give us the value.

a)
```
static void someFunction(int a, int b){
    int sum=0
    for(int i=0; i<a; i++) // runs a times
        for(int j=0; j<b; j++) // runs b times
            sum += i+b; // O(1)
```

The outer loop runs from 0 to a-1, so a times
The inner loop runs from 0 to b-1, so b times
Since the innermost sum operation completes in constant time, we should write O(1) for it.

T = Time complexity
↳ depends on a and b

$$T(a,b) = \sum_{i=0}^{a-1} \sum_{j=0}^{b-1} O(1)$$

$$T(a,b) = \sum_{i=0}^{a-1} b(O(1))$$

$$T(a,b) = a.b.O(1)$$

$$\boxed{T(a,b) = O(a \cdot b)}$$

b)
```
static void anotherFunction(int a){
    int sum=0; ------------------> O(1)
    for(int i=0; i<a; i++)
        sum += i; ---------------> O(1)
        i = i*2;
```

Here we need to find out how many times the for loop is used so;
Let for loop be used x times.

$$2^x = a \Rightarrow x = \log_2 a$$

And the assignment operation inside is O(1)

$$T(a) = \sum_{i=0}^{x} O(1) \Rightarrow \sum_{i=0}^{\log_2 a} O(1)$$

$$= T(a) = \log_2 a\, O(1)$$

$$\boxed{T(a) = O(\log_2 a)} \longrightarrow \text{approximately } T(a) = O(\log a)$$

Fatih Emre Ozgan
23010400 4090

(c)
```
static void differentFunction(int n) {
    for(int i=0; i<n; i++)
        for(int j=0; j<n; j++)
            for(int k=0; k<n; k++)
                System.out.println("Hello World!"); // O(1)
}
```

In this example all loops run n times. And the print function inside is O(1).

T → Time complexity
↳ depend on n

$$T(n) = \sum_{i=0}^{n} \sum_{j=0}^{n} \sum_{k=0}^{n} O(1)$$

$$T(n) = \sum_{i=0}^{n} \sum_{j=0}^{n} n\,O(1)$$

$$T(n) = \sum_{i=0}^{n} n^2\,O(1)$$

$$T(n) = n^3\,O(1)$$

$$\boxed{T(n) = O(n^3)}$$

(4) In this question we have an infinite number of toys, so our algorithm will not be limited in terms of toys and we need the determine at what floor level in a 100-story building the toys start to break.

Fatih Emre Ogan
23010400409

The worst case of this problem is to find the floor by dropping toys one by one starting from one floor, but this would not be a sustainable solution. For example, if the building had 10,000 floors, it would be more difficult to solve it one by one, so we need to prepare a more logical algorithm for this problem.

I think this problem can be solved much more efficiently with the binary search method. In the binary search method, we start searching from the middle of elements we have. If the toy breaks on this floor, the breaking point must be on one of the floors below this floor, so we continue searching in the lower half. If the toy does not break, the breaking point must be on a higher floor, and we continue searching in the upper half. We continue this process by halving the search area at each step. Thus, even in the worst case we can find the right floor with very few tries.

The time equivolent of this problem;

$$T(n) = O(\log_2 n)$$

This is because "Binary search splits the search space in half at each step, so each splitting operation is done in base $\log_2(n)$. This means that at each step the space is halved and the time complexity of the operation grows logorithmically.

Usually $\log(n)$ is used in Big-O notation because the difference between the two bases does not affect the growth rate.

$T(n) = O(\log_2 n)$ (specific)

$T(n) = O(\log n)$ (usually)

Based on this algorithm, our worst case dropped from 100 to 7.